
Day 6: Building End-to-End Projects Using an Agent Harness

AI Coding Techniques · [deboistech](#)

- Moving beyond simple code completion to autonomous agent orchestration
- Core components: Planning, Tools (MCP), Sub-Agents, Custom Skills
- Constraint: Single clean pass execution to keep token consumption light

Phase 1: The Planning Phase & Harness Modes

Why Planning Saves Re-work

- Never jump straight into unstructured code edits
- Use Plan Mode or brainstorming skills before touching files
- Key decisions: Extraction strategy, summary structure, diagram format
- Rule: A 10-minute structured plan avoids 2 hours of refactoring later

Phase 2: Model Context Protocol (MCP) Integration

Connecting the Harness to the Real World

- MCP provides standardized stdio/HTTP interfaces for tool augmentation
- Enables harnesses to call external parsers (e.g., PDF extraction, diagram validation)
- Command: `opencode mcp add <name> -- <command>`
- Anti-pattern: Do not bolt on fake MCP tools; solve a real workflow need

Phase 3: Multi-Agent Architecture & Sub-Agents

Role Specialization & Isolated Context

- Primary Agent (Orchestrator): Handles main workflow, calls MCP tools, synthesizes recap
- Sub-Agent (Reviewer): Dedicated QA auditor with an independent checklist
- Defined via `opencode.json` or `.opencode/agents/recap-reviewer.md`
- Verifies factual fidelity against source slides and checks diagram syntax

Phase 4: Custom Agent Skills

Teaching Your Harness Reusable Playbooks

- Skills provide procedural knowledge and formatting rules across projects
- Stored in `.opencode/skills/<skill-name>/SKILL.md`
- Structure: YAML frontmatter + concept distillation rules + Mermaid diagram styling
- Allows any future slide deck to be processed with identical high quality

Summary & Best Practices for Production Agents

Day 6 Key Takeaways

- Keep credit usage light: Input -> MCP Extraction -> Summarize -> Review -> Output
- Prefer declarative Mermaid diagrams (flowchart TD, mindmap) over complex images
- Always include human-in-the-loop verification and testable artifact outputs